Mục lục

gem5 1
What is gem5 and why use it? 1
When to use gem5 and who uses it? 2
Các định nghĩa liên quan 2
Cache 2
TCM (Tightly Coupled Memory) 5
Data Space Exploration (DSE) 5
How 5
Data Pipeline Engineer 6

gem5

What is gem5 and why use it?

- gem5 là một nền tảng mô phỏng kiến trúc hệ thống máy tính, cho phép xây dựng mô hình hệ thống, chạy phần mềm trên mô hình đó và khảo sát các đặc tính hoạt động mà không cần xây dựng phần cứng thực tế. gem5 hỗ trợ các mô hình CPU và hệ thống bộ nhớ với những mức độ chi tiết khác nhau, cho phép nghiên cứu hành vi và thời gian thực thi của hệ thống theo các mô hình đã thiết lập. So với mô phỏng RTL chi tiết, gem5 thường phù hợp hơn cho việc khảo sát nhiều cấu hình kiến trúc ở cấp độ hệ thống do không cần mô phỏng toàn bộ logic phần cứng ở mức tín hiệu. Điều này giúp tiết kiệm thời gian và chi phí trong quá trình đánh giá các ý tưởng thiết kế ban đầu.
- Khi sử dụng gem5 thì sẽ chủ yếu chạy thông qua Python để cấu hình hệ thống mô phỏng, chẳng hạn như lựa chọn mô hình CPU, cấu hình cache, thiết lập bộ nhớ và xác định kết nối giữa các thành phần. Trong khi đó, phần lớn logic mô phỏng được triển khai bằng C++, bao gồm hành vi của các mô hình CPU, hệ thống bộ nhớ và cơ chế mô phỏng thời gian.
- Như vậy gem5 có thể giúp thực hiện các mục đích như:
    - Đánh giá hiệu năng và các bottleneck: gem5 có thể trả về các thông số về execution time, cache hit/miss, độ trễ,… Giúp nhận thấy các thành phần nào của hệ thống làm giảm hiệu suất để tìm ra các hướng phát triển
    - Khám phá không gian thiết kế (Design Space Exploration): Vì ta có thể sử dụng các thuật toán từ lặp trình mô phỏng để kiểm thử nhiều thiết kế khác nhau một ít chi tiết để khảo sát và đánh giá xem thiết kế nào là tối ưu, phù hợp nhất so với mục tiêu thiết kế. Một điều mà sẽ không thể làm được nếu chỉ thiết kế bằng tay.
    - Giúp nghiên cứu ở các cấp độ hệ thống: Vì trong một hệ thống máy tính, rất nhiều linh kiện giao tiếp với nhau như các CPU core, caches, memory controller, RAM,… Và khi thấy đổi cấu trúc một thành phần linh kiện này sẽ ảnh hưởng đến toàn bộ hệ thống. Thông qua gem5 thì ta có thể khảo sát những thay đổi này để nghiên cứu sâu hơn dẫn đến các nghiên cứu như hệ thống đa lõi, tối ưu hệ thống bộ nhớ, hay đánh giá sự đánh đổi giữa hiệu năng và chi phí thiết kế…
    - Thực hiện Full-system Simulation (FS) hoặc Syscall Emulation (SE)
    - Linh hoạt trong các phương pháp mô phỏng: Có thể tùy chỉnh các chế độ mô phỏng như trên, ngoài ra còn cho phép sử dụng và chỉnh sửa hay tự tạo nhiều mô hình linh kiện khác nhau, như loại CPU, caches, board,… để mô phỏng.

- Có thể thiết kế tính nhất quán của caches: Do ta có thể tùy chỉnh, tự tạo các mô hình cache riêng cũng như cách nó hoạt động, giao thức của chúng.
- Hỗ trợ nghiên cứu nhiều ISA: gem5 hỗ trợ nhiều loại ISA như RISC-V, x86, ARM nên cho phép mô phỏng và so sánh các ISA với nhau trong cùng một framework mô phỏng.

When to use gem5 and who uses it?

- Qua những đặc điểm trên, có thể thấy một trong những ưu điểm quan trọng của gem5 là khả năng khảo sát và đánh giá các ý tưởng thiết kế kiến trúc máy tính trước khi triển khai phần cứng. Chẳng hạn, nếu muốn tăng dung lượng L2 cache hoặc thay đổi associativity, ta có thể cấu hình các phương án tương ứng trong gem5 rồi chạy cùng một workload để so sánh hiệu năng. Nhờ đó, người thiết kế có thể đánh giá sơ bộ lợi ích và hạn chế của từng phương án trước khi quyết định có nên tiếp tục đầu tư vào quá trình triển khai phần cứng hay không. Tuy nhiên, kết quả mô phỏng phụ thuộc vào độ chính xác của các mô hình và không thay thế hoàn toàn việc kiểm chứng trên phần cứng thực tế.
- Như vậy Đối tượng sử dụng gem5 thường bao gồm các kiến trúc sư máy tính, kỹ sư kiến trúc vi xử lý và các nhà nghiên cứu về hiệu năng hệ thống. Trong quy trình phát triển phần cứng, gem5 có thể được sử dụng trong giai đoạn nghiên cứu và đánh giá kiến trúc, hỗ trợ đưa ra quyết định trước hoặc song song với quá trình phát triển RTL. Sau đó mới thiết kế phần cứng vẫn cần trải qua các bước phù hợp như verification, synthesis và physical design để kiểm tra tính đúng đắn và đánh giá khả năng triển khai thực tế.

Các định nghĩa liên quan

Cache

- Gem5 là công cụ mô phỏng để thực hiện đề tài nhưng quan trọng của đề tài là khảo sát đánh giá cache cho nên phải hiểu biết về cache.
- Cache là toàn bộ hệ thống nhớ SRAM tốc độ cao đặt sát CPU, lưu trữ tạm thời các khối dữ liệu thường dùng từ DRAM để CPU không phải chờ đợi lâu.
- Một số thành phần của cache là:
    - Bank: Cache lớn thường không làm thành một khối duy nhất mà xẻ nhỏ thành nhiều khối phần cứng độc lập gọi là banks. Mỗi banks sẽ có cổng giải mã và đường đọc/ghi riêng cho phép CPU truy xuất song song nhiều dữ liệu cùng một lúc mà không bị nghẽn (dữ liệu phải nằm ở các bank khác nhau).
    - Set: Set là một hàng dữ liệu trong Bank, được chọn đích danh bởi trường Set Index trong địa chỉ CPU phát ra. Khi CPU cần tìm một địa chỉ, nó chỉ được

phép tìm trong đúng duy nhất 1 Set tương ứng với Index đó mà không được tìm ở Set khác.

- Way: là các ngăn chưa dữ liệu song song nằm trong cùng một Set. Số lượng Way trong 1 Set chính là độ kết hợp (Associativity) cho phép nhiều địa chỉ khác nhau (nhưng có trùng Set Index) có thể cùng chung sống trong một Set mà không đá văng nhau ra ngoài.
- Cache Line/Block: là đơn vị lưu trữ cơ bản nhỏ nhất trong Cache. Mỗi một Way là một Cache Line hoàn chỉnh.
  Một Cache Line gồm 4 thành phần (tùy vào thiết kế có thể là 3):
  - Valid bit: 1 bit báo dữ liệu này có hợp lệ hay không (1: có dữ liệu, 0: đang trống)
  - Dirty bit: 1 bit báo dữ liệu trong Cache đã bị CPU sửa đổi chưa (1: đã sửa, 0: chưa sửa)
  - Tag Store: lưu các bit địa chỉ cao để nhận diện dữ liệu này thuộc địa chỉ nào của RAM.
  - Data Block (64 Byte): khối dữ liệu thật lấy từ DRAM lên, được chia thành 16 Words.

- Cache Hit: Khi CPU tìm trong Set tương ứng, có ít nhất một Way chưa Tag trùng khớp với Tag của địa chỉ CPU yêu cầu và valid bit bằng 1 thì là cache hit. Tức là tìm thấy được dữ liệu yêu cầu từ CPU trong cache.
- Cache Miss: Khi CPU tìm trong Set tương ứng mà không có Way nào khớp Tag, hoặc có tìm đúng mà valid bit bằng 0 thì tính là Cache Miss. Tức là không tìm thấy được dữ liệu yêu cầu từ CPU trong cache. Có 3 loại Cache miss chính:
  - Compulsory Miss: Khi dữ liệu yêu cầu chưa từng được truy cập đến nên trên Cache không có dữ liệu này dẫn đến Cache miss. Lúc này có tăng kích thước cache lên hay không vẫn không tránh được loại miss này.
  - Capacity Miss: Khi cache quá nhỏ và tất cả dữ liệu cần thiết không chứa đủ trong cache dẫn đến các dữ liệu cũ bị loại bỏ để ghi thay cho dữ liệu mới. Lúc này tăng kích thước cache lên có thể giúp khắc phục miss này.
  - Conflict Miss: Khi các khối dữ liệu được gắn cho cùng một Set trên cache và cạnh tranh các Way trên Set đó thì sẽ dẫn đến Conflict Miss do có thể khối này phải đá khối kia ra khỏi Set mặc dù trên Cache vẫn còn các Set trống khác. Lúc này tăng độ kết hợp Associativity lên có thể giúp khắc phục.
- Ví dụ: địa chỉ mẫu từ CPU 0x0000A084 qua binary là 0b0000 0000 0000 0000 1010 0000 10 0001 00

    - Tag [31:12]: là phần 0x0000A, dùng để so sánh song song với 16 Tag trong Set và kiểm tra valid bit. Quyết đinh cache hit hay miss.
    - Set Index [11:6]: là phần 0b000010, sẽ đi qua Set Row Decoder để biết dữ liệu nằm trong set nào trong 64 Set để so sánh tag, kích hoạt đông thời cả 16 Ways trong Set được chọn.
    - Word Offset [5:2]: là phần 0001, giúp chọn 1 trong 16 words trong Data Block của Cache line.
    - Byte Offset [1:0]: là phần cuối 00, giúp chọn 1 trong 4 bytes của word được chọn từ Word Offset.
- Cache Read: Như vậy với Cache Miss thì từng lớp Cache có thể Miss riêng biệt, ví dụ dữ liệu cần truy cập không nằm trong L1 thì L1 Miss nhưng có nằm ở L2 thì L2 Hit. Một quá trình Read của CPU có thể là CPU → L1 Miss → L2 Miss → DRAM, rồi từ DRAM thì dữ liệu sẽ ghi lại lần lượt trên L2 và L1 và trả về cho CPU. Và nếu như Cache đầy và phải loại bỏ dữ liệu ra để cho dữ liệu mới vào thì phụ thuộc vào replacement protocol của thiết kế như LRU là least recently used loại bỏ dữ liệu được sử dụng cũ nhất, hay FIFO,…
- Cache Write: CPU sẽ cặp nhật dữ liệu như thế nào nếu như thay đổi dữ liệu? Ví dụ có A=10 nằm trên L1, L2 và DRAM. Lúc này nếu CPU cập nhật A=20 thì sẽ có 2 cách cập nhật:
    - Write-Through: Dữ liệu mới được cập nhật sẽ được cập nhật liền cho cả L1, L2 và DRAM. Tức là CPU → L1 A=20 update → L2 A=20 update → DRAM A=20 update. Tuy nhiên cách này dẫn đến nhiều lượng giao thông dữ liệu có thể dẫn đến bottleneck.
    - Write-Back: cách này giúp khắc phục nhược điểm của Write-Through là chỉ cập nhật khi cần thiết. Ví dụ CPU → L1 =A20 update thì tại Cache Line này sẽ đặt Dirty bit bằng 1. Sau khi Cache loại bỏ Cache line này thì thấy rằng Dirty bit bằng 1 mới cập nhật cho L2 A=20 rồi mới loại bỏ Cache Line đó ở dưới L1. Tương tự vậy cho L2 đến DRAM nếu L2 loại bỏ Cache Line chứa A.
- Cách đánh giá Cache:
    - Hit Rate: Là tỉ lệ Cache Hit trên toàn bộ lần truy cập dữ liệu có công thức là Cache Hit / Total Cache Accesses
    - Miss Rate: Là tỉ lệ Cache Miss trên toàn bộ lần truy cập dữ liệu có công thức là Cache Miss / Total Cache Accesses
    - Ngoài ra Hit Rate và Miss Rate tổng ra thì từng tầng lớp Cache có thông số riêng là Local Miss Rate và Global Miss Rate. Nếu như ta truy cập dữ liệu 1000 lần và trong số 1000 lần đó thì L1 miss 100 lần như vậy Local của L1 là

100/1000, Còn sau khi L1 miss mới qua tới L2 thì trong 100 lần cập nhật L2 này thì L2 chỉ Miss 20 lần như vậy Local của L2 là 20/100 nhưng global lại là 20/1000.

- AMAT (Average Memory Access Time): là trung bình thời gian truy cập bộ nhớ có công thức $AMAT = T_{L1} + MR_{L1}T_{L2} + MR_{L1}MR_{L2}T_{DRAM}$.

TCM (Tightly Coupled Memory)

- Thì như trên có được định nghĩa của Cache xin ra được vấn đề là hiệu suất của Cache khó lường được đối với các lượng công việc Workload khác nhau, vì vậy mà khi ta tối ưu Cache là ta tối ưu trung bình hiệu suất của nó.
- Như vậy mới có khối TCM được chia thành ITCM (Instruction TCM) và DTCM (Data TCM). Các khối này có vùng riêng của nó và đặt kế CPU. Đối với Cache các dữ liệu khi được đọc và ghi là do phần cứng quản lý như khối dữ liệu này được đặt vào Set nào, Way nào là đều do phần cứng. Nhưng TCM là khối có độ trễ rất thấp và được quản lý bằng tay (qua phần mềm, compiler hay thiết kế của hệ thống) dùng để đặt các phần code mà ta cần chạy rất nhanh, hay các dữ liệu thường xuyên truy cập, hay là bộ nhớ stack,… Vì vậy những dữ liệu quan trọng đều được đặt trong TCM và có thể lường trước được độ trễ của nó và tối ưu độ trễ này.
- Tóm lại Cache lưu trữ dữ liệu nào là phụ thuộc vào replacement protocol (LRU, random, FIFO, …), còn TCM thì ta tự tay đặt các lệnh, dữ liệu quan trọng/nhạy cảm với độ trễ, có thể được hưởng lợi từ việc truy cập rất nhanh và lường trước được vào để luôn có thể cập nhật nhanh mà không sợ Cache Miss.

Data Space Exploration (DSE)

- DSE là quá trình đi kiểm thử nhiều cấu hình khác nhau và khảo sát ảnh hưởng của chúng đối với hiệu năng mô hình, thì trong đề tài này em sẽ chủ yếu thay đổi kích thước L2, thay đổi Associativity của của Cache từ 2 đến 16, hay thay đổi kích thước của từng Cache Line. Và với mỗi thay đổi này thì sẽ thử nghiêm các hỗn hợp của nó lấy ra các thông số quan trọng và xem cấu hình nào là tốt nhất.

How

- Để tiến hành làm phần này thì em sẽ phải tiếp tục làm quen với gem5. Hiện tại gem5 em chỉ mới chạy file python với standardlib thông qua hướng dẫn khóa gem5 bootcamp trên youtube. Thì sẽ cần phải tự mình thử nghiệm nhiều hơn. Ngoài ra tìm hiểu các thông số trong stats.txt.
- Tất nhiên là quá trình kiểm thử nhiều cấu hình này phải tự động hóa chứ không phải chạy tay từng mô phỏng của từng cấu hình nên em sẽ phải học và viết một file python script để chạy toàn bộ.

Data Pipeline Engineer

- Sau quá trình mô phỏng trên gem5, em sẽ xây dựng các script để tự động thu thập và lọc các thông số mô phỏng quan trọng cần đánh giá, chẳng hạn như thời gian thực thi, IPC, Cache Miss Rate và các thông số liên quan đến L2 Cache. Các dữ liệu này sẽ được tổng hợp thành bảng để thuận tiện cho việc so sánh giữa các cấu hình khác nhau. Từ kết quả thu được, em sẽ hỗ trợ phân tích và đánh giá ảnh hưởng của các tham số L2 Cache đến hiệu suất hệ thống.